C2 beadandó 2. fázis

Neptun kód:	**CWQSKI**	Név:	**Nagy Edömér Tamás**
Beadás verziószáma:	1.		

Feladat

Programozási tételek összeépítése *

Mindenhol pontot szerző versenyzők

Egy olimpiai versenyszámban N versenyző szeretne az olimpián indulni. Számukra 5 kvalifikációs versenyt szerveztek, melynek eredményeit ismerjük. Minden versenyen az első K helyezett kap pontot, a K. helyezett 1, az előző 3, az őt megelőző 5 pontot és így tovább, tehát 2 ponttal növekszik a pontszám helyezésenként. Az olimpiára a pontszám szerint legjobb K versenyző kap indulási jogot, de holtverseny esetén ez lehet K-nál több is (a K. helyen álló versenyzővel holtversenyben levők mindegyike kijut az olimpiára).

Írj programot, amely megadja azokat a versenyzőket, akik minden versenyen szereztek pontot!

Bemenet

A *standard bemenet* első sora a versenyzők számát és a K számot ($2 \le K \le N \le 100$) tartalmazza. A következő N sorban egy-egy versenyző 5 versenyen elért helyezése található (0, ha nem indult, vagy $1 \le H_{i,j} \le N$). A helyezésekre teljesül, hogy ha H a legnagyobb helyezés, akkor minden 1 és H közötti érték egyszer fordul elő.

Kimenet

A *standard kimenet* első sorába a minden versenyen pontot szerzett versenyzők M számát kell írni! A második sorba az ilyen versenyzők sorszámai kerüljenek, ha többen vannak, akkor sorszám szerint növekvő sorrendben (ha egyetlen ilyen versenyző sincs, akkor az első sorba 0 kerüljön, a második sor üres legyen, de ekkor is két sorból kell állnia a kimenetnek)!

Példa

Bemenet	Kimenet
6 4 1 4 1 3 2 5 5 0 0 5 2 1 2 2 1 3 2 3 4 3 0 3 0 1 0 4 0 0 0 4	3 1 3 4

Korlátok

Időlimit: 0.1 mp.

Memórialimit: 32 MiB

Specifikáció

```
Be: n∈Z,
    k∈Z,
    versenyzok∈Z[1..n,1..5]
Ki: db∈Z,
    mindreszvett∈Z[1..db]

Fv: mindpont: Z[,] x Z x Z -> L,
    mindpont(vk, k, ind) = MIND(
            j=1..5,
            vk[ind, j] > 0 és vk[ind, j] <= k
        )

Ef: 2≤k≤n≤100 és
    MIND(
        i=1..n,
        MIND(
            j=1..5,
            0 ≤ versenyzok[i, j] ≤ DARAB(
                k=1..n,
                versenyzok[k, j] ≠ 0
            ) és
            (
                DARAB(
                    k=1..n,
                    versenyzok[i, j] = versenyzok[k, j]
                ) = 1 vagy
                versenyzok[i, j] = 0
            )
        )
    )
Uf: (db, mindreszvett)=KIVÁLOGAT(
        i=1..n,
        mindpont(versenyzok, k, i),
        i
    )
```

Sablon

külső algoritmus sablonja:

Kiválogatás
sablon

i		T(i)		f(i)
e	→	HAMIS		
e+1	→	IGAZ	→	1 f(e+1)
e+2	→	IGAZ	→	2 f(e+2)
u	→	HAMIS		

Feladat

Adott az egész számok egy [e..u] intervalluma, egy ezen értelmezett T:[e..u]→Logikai feltétel és egy f:[e..u]→H függvény. Határozzuk meg az f függvény az [e..u] intervallum azon értékeinél felvett értékeit, amelyekre a T feltétel teljesül!

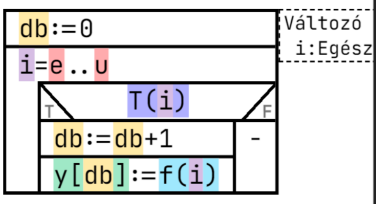
Specifikáció

```
Be: e∈Z, u∈Z
Ki: db∈N, y∈H[1..db]
Ef: -
Uf: db=DARAB(i=e..u,T(i)) és
    ∀i∈[1..db]:(
      ∃j∈[e..u]:T(j) és y[i]=f(j))
 és y⊆(f(e),f(e+1),...,f(u))
```

Rövidítve:

```
Uf: (db,y)=KIVÁLOGAT(i=e..u,T(i),f(i))
```

Algoritmus



belső algoritmus sablonja::

Mind eldöntés (vagy optimista eldöntés)
sablon

Feladat

Adott az egész számok egy [e..u] intervalluma és egy T:[e..u]→Logikai feltétel. Határozzuk meg, hogy az [e..u] intervallumnak mindegyik eleme olyan-e, amely kielégíti a T feltételt!

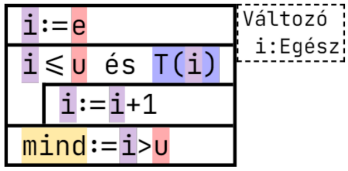
Specifikáció

```
Be: e∈Z, u∈Z
Ki: mind∈L
Ef: -
Uf: mind=∀i∈[e..u]:(T(i))
```

Rövidítve:

```
Uf: mind=MIND(i=e..u,T(i))
```

Algoritmus



Visszavezetés

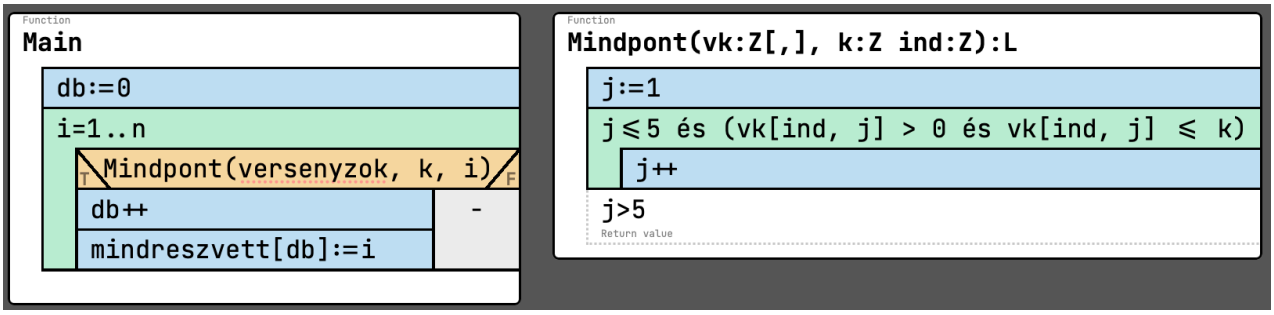
Külső algoritmus: KIVÁLOGAT

(db, y)	~	(db, mindreszvett)
i=e..u	~	i=1..n
T(i)	~	mindpont(versenyzok, k, i)
f(i)	~	i

Belső algoritmus: MIND

mind	~	mindpont(vk, k, ind)
i=e..u	~	j=1..5
T(i)	~	vk[ind, j] > 0 és vk[ind, j] <= k

Algoritmus



Kód (C#)

```
/// Nagy Edömér Tamás CWQSKI
// edomer@student.elte.hu

using System;

namespace code
{
    class Program
    {

        static void Main(string[] args)
        {
            // Deklaráció

            const int MAX_N = 100;
            int n, k, db, i, j, prev;
            int[,] versenyzok = new int[MAX_N, 5];
            int[] mindreszvett = new int[MAX_N];
            string[] linesplit;
            bool valid, valid2;
            int[] v_column;

            // Beolvasás
            n = 0; k = 0; // Hogy a fordító ne sírjon
            do
            {
                linesplit = Console.ReadLine().Split();
                valid = linesplit.Length == 2;
                if (valid)
                {
                    valid = int.TryParse(linesplit[0], out n);
                    valid = valid && int.TryParse(linesplit[1], out k);
                    valid = valid && (2 <= k && k <= n && n <= MAX_N);
                }
                if (!valid)
                          Console.Error.WriteLine("Nem sikerült beolvasni a
megadott 'n k' értékeket. Próbáld újra.");
            } while (!valid);


            v_column = new int[n];

            do
            {
                for (i = 0; i < n; ++i)
                {
```

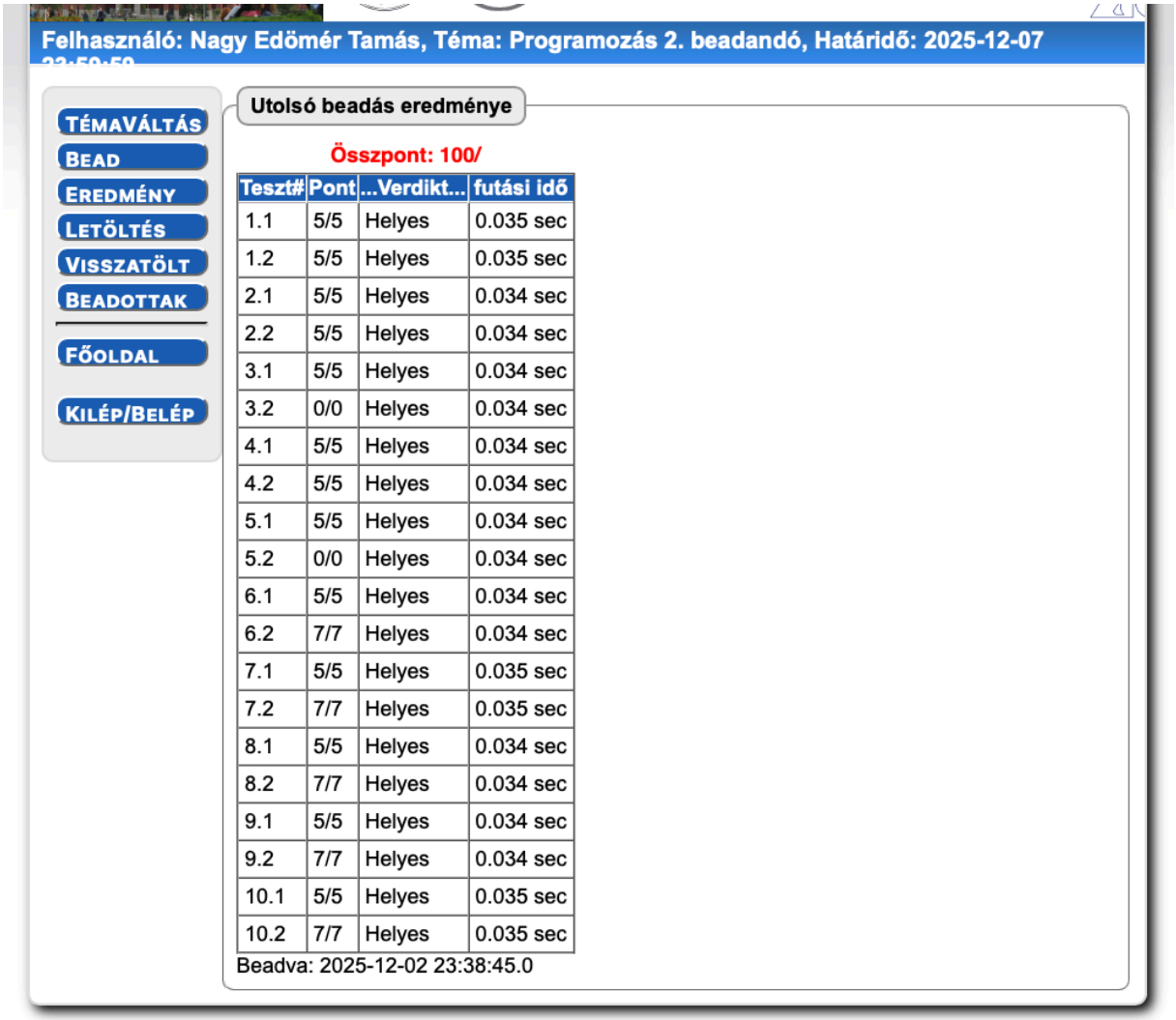
```
                    do
                    {
                        linesplit = Console.ReadLine().Split();
                        valid = linesplit.Length == 5;
                        if (valid)
                        {
                            for (j = 0; j < 5; ++j)
                            {
                                valid = valid && int.TryParse(linesplit[j],
out versenyzok[i, j]);
                                 valid = valid && (0 <= versenyzok[i, j] &&
versenyzok[i, j] <= MAX_N);
                            }
                        }
                        if (!valid)
                             Console.Error.WriteLine("Nem jó a sor bemenete!
Próbáld újra");
                    } while (!valid);
                }

                  // Annak ellenőrzése, hogy mindegyik oszopban minden 1..h
szám pontosan egyszer szerepel

                valid2 = true;
                for (i = 0; i < 5; ++i)
                {
                    // 1. put column into 1 dimensional array
                    // 2. sort array
                    // 3. check if every non-zero is +1 of the previous
                    for (j = 0; j < n; ++j)
                    {
                        v_column[j] = versenyzok[j, i];
                    }

                    Array.Sort(v_column);

                    prev = 0;
                    for (j = 0; j < n && valid2; ++j)
                    {
                        if (v_column[j] != 0)
                        {
                            if (v_column[j] == prev + 1)
                            {
                                ++prev;
                            }
                            else
                            {
                                valid2 = false;
                            }
                        }
```

```
                    }
                }
                if (!valid2)
                        Console.Error.WriteLine("Nem megfelelők a megadott versenyzők adatai! Minden versenyen pontosan 1 darab [1..h] számnak szabad lennie! Próbáld újra.");
            } while (!valid2);


            // Feldolgozás

            db = 0;
            for (i = 0; i < n; ++i)
            {
                if (Mindpont(versenyzok, k, i))
                {
                    mindreszvett[db] = i;
                    ++db;
                }
            }



            // Kiírás
            if (db != 0)
            {
                        Console.Error.Write("Minden versenyen pontot szerző versenyzők:\n - száma: ");
                Console.WriteLine(db);
                Console.Error.Write(" - indexei: ");
                for (i = 0; i < db; ++i)
                {
                    Console.Write("{0} ", mindreszvett[i] + 1);
                }
                Console.WriteLine();
            }
            else
            {
                Console.Error.Write("Nincs ilyen versenyző! (");
                Console.Write("0");
                Console.Error.Write(")");
                Console.WriteLine();
                Console.WriteLine();
            }

        }

        static bool Mindpont(int[,] vk, int k, int ind)
        {
```

```
            int j = 0;
            while (j < 5 && (vk[ind, j] > 0 && vk[ind, j] <= k))
                ++j;
            return j==5;
        }
    }
}
```

Bíró pontszám és képernyőkép

100 pont



Felhasználó: Nagy Edömér Tamás, Téma: Programozás 2. beadandó, Határidő: 2025-12-07

TÉMAVÁLTÁS
BEAD
EREDMÉNY
LETÖLTÉS
VISSZATÖLT
BEADOTTAK
FŐOLDAL
KILÉP/BELÉP

Utolsó beadás eredménye

Összpont: 100/

Teszt#	Pont	...Verdikt...	futási idő
1.1	5/5	Helyes	0.035 sec
1.2	5/5	Helyes	0.035 sec
2.1	5/5	Helyes	0.034 sec
2.2	5/5	Helyes	0.034 sec
3.1	5/5	Helyes	0.034 sec
3.2	0/0	Helyes	0.034 sec
4.1	5/5	Helyes	0.034 sec
4.2	5/5	Helyes	0.034 sec
5.1	5/5	Helyes	0.034 sec
5.2	0/0	Helyes	0.034 sec
6.1	5/5	Helyes	0.034 sec
6.2	7/7	Helyes	0.034 sec
7.1	5/5	Helyes	0.035 sec
7.2	7/7	Helyes	0.035 sec
8.1	5/5	Helyes	0.034 sec
8.2	7/7	Helyes	0.034 sec
9.1	5/5	Helyes	0.034 sec
9.2	7/7	Helyes	0.034 sec
10.1	5/5	Helyes	0.035 sec
10.2	7/7	Helyes	0.035 sec

Beadva: 2025-12-02 23:38:45.0

Saját tesztfájlok

Bemenet	Kimenet
5 4 1 4 1 3 2 4 5 0 0 4 2 1 2 2 1 3 2 3 4 3 0 3 0 1 0	Minden versenyen pontot szerző versenyzők: - száma: 3 - indexei: 1 3 4
4 2 1 2 1 1 1 2 1 2 2 2 3 0 3 3 3 0 0 0 0 4	Minden versenyen pontot szerző versenyzők: - száma: 2 - indexei: 1 2
4 4 1 4 1 3 2 4 3 0 0 4 2 1 2 2 1 3 2 3 1 3	Minden versenyen pontot szerző versenyzők: - száma: 3 - indexei: 1 3
4 3 0 1 1 1 1 0 2 2 2 2 0 3 3 3 3 0 4 4 4 4	Nincs ilyen versenyző! (0)
Hibás tesztesetre példa: 6 4 6 4 1 3 2 5 5 0 0 5 2 1 2 2 1 3 2 3 4 3 0 3 0 1 0 4 0 0 0 4	Nem megfelelők a megadott versenyzők adatai! Minden versenyen pontosan 1 darab [1..h] számnak szabad lennie! Próbáld újra.